Codice sorgente del modulo telnet_version

Il codice sorgente del modulo `telnet_version` è scritto in Ruby, linguaggio utilizzato da Metasploit perché permette di sviluppare moduli in modo abbastanza semplice e integrarli con le funzionalità già presenti nel framework.

Andando a documentarsi e osservando il codice tramite il comando `edit`, si vede che il modulo viene definito come:

```
class MetasploitModule < Msf::Auxiliary
```

Questo indica che il modulo è di tipo auxiliary, quindi viene utilizzato per attività di analisi. Subito dopo viene inclusa la libreria:

```
include Msf::Exploit::Remote::Telnet
```

che permette al modulo di comunicare tramite protocollo Telnet, e:

```
include Msf::Auxiliary::Scanner
```

che consente di eseguire il modulo su uno o più host.

Nella fase di inizializzazione vengono definiti alcuni parametri, tra cui:

```
Opt::RPORT(23)
```

che imposta la porta 23, utilizzata dal servizio Telnet, e:

```
OptInt.new('TIMEOUT', [true, 'Timeout for the Telnet probe', 30])
```

Il parametro TIMEOUT indica il tempo massimo (in secondi) che il modulo attende una risposta dal server. Se entro questo tempo il server non risponde, la connessione viene interrotta per evitare che il programma rimanga bloccato.

Analizzando la parte principale del codice, si trova la funzione:

```
def run_host(ip)
```

che viene eseguita per ogni macchina target. All'interno di questa funzione il modulo stabilisce una connessione tramite:

```
res = connect
```

Una volta aperta la connessione, il modulo legge il banner del server e lo rende leggibile con:

```
banner_sanitized = Rex::Text.to_hex_ascii(banner.to_s)
```

In questo caso il termine "sanitizzare" indica la trasformazione del banner in un formato leggibile, eliminando o convertendo eventuali caratteri non stampabili.

Successivamente, il risultato viene mostrato a schermo tramite:

```
print_good("#{ip}:#{rport} TELNET #{banner_sanitized}")
```

e registrato all'interno di Metasploit con:

```
report_service(...)
```

Infine, il codice gestisce eventuali errori tramite istruzioni come:

```
rescue ::Rex::ConnectionError
rescue Timeout::Error
```

che permettono di gestire problemi di connessione o mancata risposta del server senza interrompere l'esecuzione del modulo.

Nel complesso, dal codice si capisce che il modulo si connette al servizio Telnet, legge il banner del server, lo rende leggibile e lo mostra a schermo, fornendo informazioni utili nella fase di analisi del sistema target.

```ruby
##
# This module requires Metasploit: https://metasploit.com/download
# Current source: https://github.com/rapid7/metasploit-framework
##

class MetasploitModule < Msf::Auxiliary
  include Msf::Exploit::Remote::Telnet
  include Msf::Auxiliary::Scanner
  include Msf::Auxiliary::Report

  def initialize
    super(
      'Name' => 'Telnet Service Banner Detection',
      'Description' => 'Detect telnet services',
      'Author' => 'hdm',
      'License' => MSF_LICENSE
    )
    register_options(
      [
        Opt::RPORT(23),
        OptInt.new('TIMEOUT', [true, 'Timeout for the Telnet probe', 30])
      ]
    )
  end

  def to
    return 30 if datastore['TIMEOUT'].to_i.zero?

    datastore['TIMEOUT'].to_i
  end

  def run_host(ip)
    begin
      ::Timeout.timeout(to) do
        res = connect
        # This makes db_services look a lot nicer.
        banner_santized = Rex::Text.to_hex_ascii(banner.to_s)
        print_good("#{ip}:#{rport} TELNET #{banner_santized}")
        report_service(:host => rhost, :port => rport, :name => "telnet", :info => banner_santized)
      end
    rescue ::Rex::ConnectionError, ::Errno::ECONNRESET => e
      print_error("A network issue has occurred: #{e.message}")
      elog("A network issue has occurred", error: e)
    rescue Timeout::Error => e
      print_error("#{target_host}:#{rport}, Server timed out after #{to} seconds. Skipping.")
      elog("#{target_host}:#{rport}, Server timed out after #{to} seconds. Skipping.", error: e)
    rescue ::Exception => e
      print_error("#{e} #{e.backtrace}")
      elog(e)
    end
  end
end
```

Codice sorgente del modulo telnet_login

Come nel caso del modulo precedente, anche `telnet_login` è scritto in Ruby e segue la stessa struttura dei moduli Metasploit già analizzati. In questo caso, però, il codice implementa direttamente la logica di autenticazione al servizio Telnet.

Osservando il codice tramite il comando `edit`, si nota che vengono incluse diverse librerie aggiuntive, tra cui quelle per la gestione delle credenziali e delle sessioni. Questo è coerente con lo scopo del modulo, che non si limita a leggere informazioni ma prova ad effettuare il login.

Nella parte centrale del codice si trova la funzione principale:

```
def run_host(ip)
```

All'interno di questa funzione vengono preparate le credenziali da utilizzare:

```
cred_collection = build_credential_collection(
 username: datastore['USERNAME'],
 password: datastore['PASSWORD']
)
```

Questa parte prende username e password inseriti dall'utente e li prepara per essere utilizzati durante il tentativo di autenticazione.

Successivamente viene creato uno scanner Telnet:

```
scanner = Metasploit::Framework::LoginScanner::Telnet.new(
```

Qui il modulo configura una connessione verso il target (`host: ip`, `port: rport`) e imposta diversi parametri, tra cui timeout e velocità dei tentativi. Questo oggetto gestisce concretamente il processo di login.

Il modulo avvia quindi la scansione con:

```
scanner.scan do |result|
```

Per ogni risultato ottenuto, il codice verifica se il login è riuscito:

```
if result.success?
```

In caso positivo, le credenziali vengono salvate e viene stampato il messaggio di successo:

```
print_good("#{ip}:#{rport} - Login Successful")
```

Subito dopo, se previsto, viene avviata una sessione Telnet:

```
start_telnet_session(ip, rport, result.credential.public, result.credential.private, scanner)
```

Questa funzione permette di aprire una connessione interattiva con il sistema target.

In caso di errore o credenziali non valide, il modulo stampa un messaggio di fallimento:

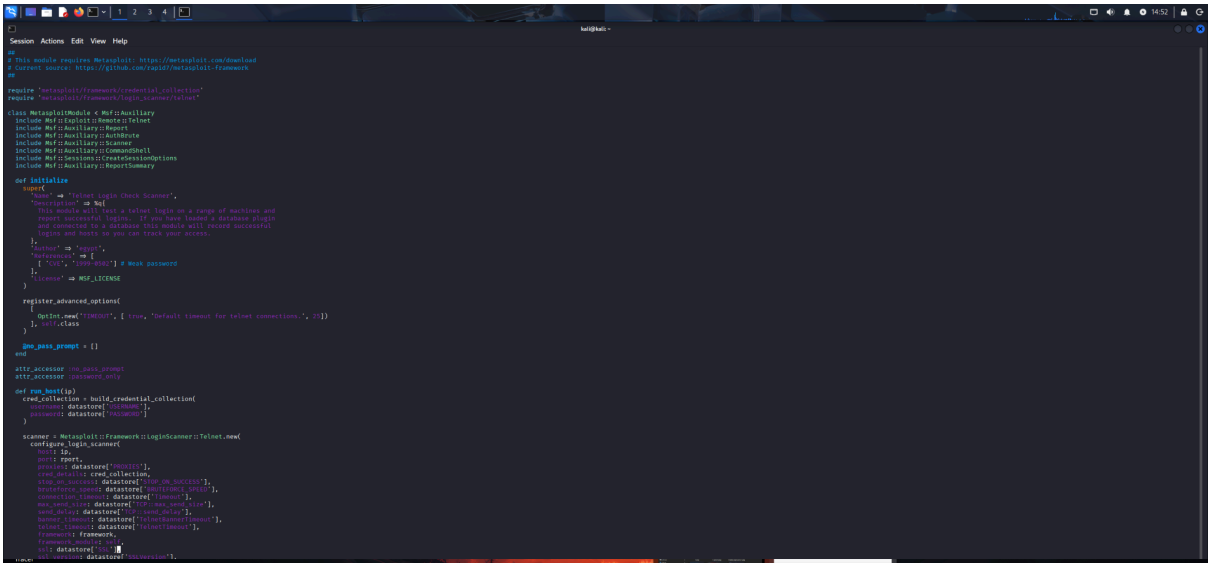
```
vprint_error("#{ip}:#{rport} - LOGIN FAILED")
```

Infine, nella funzione:

```
def start_telnet_session(...)
```

viene gestita la creazione della sessione, utilizzando username e password per stabilire una connessione remota al sistema.

Il modulo prende le credenziali inserite, tenta l'autenticazione sul servizio Telnet e, in caso di successo, apre una sessione che permette di eseguire comandi sulla macchina target.



```ruby
##
# This module requires Metasploit: https://metasploit.com/download
# Current source: https://github.com/rapid7/metasploit-framework
##

require 'metasploit/framework/credential_collection'
require 'metasploit/framework/login_scanner/telnet'

class MetasploitModule < Msf::Auxiliary
  include Msf::Exploit::Remote::Telnet
  include Msf::Auxiliary::Report
  include Msf::Auxiliary::AuthBrute
  include Msf::Auxiliary::Scanner
  include Msf::Auxiliary::CommandShell
  include Msf::Sessions::CreateSessionOptions
  include Msf::Auxiliary::ReportSummary

  def initialize
    super(
      'Name' => 'Telnet Login Check Scanner',
      'Description' => %q{
        This module will test a telnet login on a range of machines and
        report successful logins.  If you have loaded a database plugin
        and connected to a database this module will record successful
        logins and hosts so you can track your access.
      },
      'Author' => 'egypt',
      'References' => [
        [ 'CVE', '1999-0502'] # Weak password
      ],
      'License' => MSF_LICENSE
    )

    register_advanced_options(
      [
        OptInt.new('TIMEOUT', [ true, 'Default timeout for telnet connections:', 20])
      ], self.class
    )

    @no_pass_prompt = []
  end

  attr_accessor :no_pass_prompt
  attr_accessor :password_only

  def run_host(ip)
    cred_collection = build_credential_collection(
      username: datastore['USERNAME'],
      password: datastore['PASSWORD']
    )

    scanner = Metasploit::Framework::LoginScanner::Telnet.new(
      configure_login_scanner(
        host: ip,
        port: rport,
        proxies: datastore['PROXIES'],
        cred_details: cred_collection,
        stop_on_success: datastore['STOP_ON_SUCCESS'],
        bruteforce_speed: datastore['BRUTEFORCE_SPEED'],
        connection_timeout: datastore['Timeout'],
        max_send_size: datastore['TCP::max_send_size'],
        send_delay: datastore['TCP::send_delay'],
        banner_timeout: datastore['TelnetBannerTimeout'],
        telnet_timeout: datastore['TelnetTimeout'],
        framework: framework,
        framework_module: self,
        ssl: datastore['SSL'],
        ssl_version: datastore['SSLVersion'],
```

```ruby
  def run_host(ip)
    cred_collection = build_credential_collection(
      username: datastore['USERNAME'],
      password: datastore['PASSWORD']
    )

    scanner = Metasploit::Framework::LoginScanner::Telnet.new(
      configure_login_scanner(
        host: ip,
        port: rport,
        proxies: datastore['PROXIES'],
        cred_details: cred_collection,
        stop_on_success: datastore['STOP_ON_SUCCESS'],
        bruteforce_speed: datastore['BRUTEFORCE_SPEED'],
        connection_timeout: datastore['Timeout'],
        max_send_size: datastore['TCP::max_send_size'],
        send_delay: datastore['TCP::send_delay'],
        banner_timeout: datastore['TelnetBannerTimeout'],
        telnet_timeout: datastore['TelnetTimeout'],
        framework: framework,
        framework_module: self,
        ssl: datastore['SSL'],
        ssl_version: datastore['SSLVersion'],
        ssl_verify_mode: datastore['SSLVerifyMode'],
        ssl_cipher: datastore['SSLCipher'],
        local_port: datastore['CPORT'],
        local_host: datastore['CHOST']
      )
    )

    scanner.scan! do |result|
      credential_data = result.to_h
      credential_data.merge!(
        module_fullname: self.fullname,
        workspace_id: myworkspace_id
      )
      if result.success?
        credential_data[:private_type] = :password
        credential_core = create_credential(credential_data)
        credential_data[:core] = credential_core
        create_credential_login(credential_data)
        print_good "#{ip}:#{rport} - Login Successful: #{result.credential}"
        start_telnet_session(ip, rport, result.credential.public, result.credential.private, scanner) if datastore['CreateSession']
      else
        invalidate_login(credential_data)
        vprint_error "#{ip}:#{rport} - LOGIN FAILED: #{result.credential} (#{result.status}: #{result.proof})"
        disconnect(scanner.sock)
      end
    end
  end

  def start_telnet_session(host, port, user, pass, scanner)
    print_status "Attempting to start session #{host}:#{port} with #{user}:#{pass}"
    merge_me = {
      'USERPASS_FILE' => nil,
      'USER_FILE' => nil,
      'PASS_FILE' => nil,
      'USERNAME' => user,
      'PASSWORD' => pass
    }

    start_session(self, "TELNET #{user}:#{pass} (#{host}:#{port})", merge_me, true, scanner.sock)
  end
end
~
~
~
```